



Hack The Box

CURSO:
SEGURIDAD DEL SOFTWARE

WRITE-UP NIVEL II

Kener Adiel Castillo

June 23, 2026

Índice

1	Resumen ejecutivo	2
1.1	Alcance	2
1.2	Hallazgos y recomendaciones	2
2	Información general	2
3	Reconocimiento	3
3.1	Conexión y recopilación inicial de información	3
3.2	Elementos identificados	4
4	Enumeración y análisis	5
4.1	Hallazgos	5
4.2	Posibles vectores de ataque	6
4.3	Caminos probados	6
5	Explotación	7
5.1	Vulnerabilidades identificadas	7
5.2	Pasos para explotación	9
6	Evidencia de resolución	12
7	Remediación	12
8	Lecciones aprendidas	13
9	Recursos consultados	13

1 Resumen ejecutivo

1.1 Alcance

Este challenge de dificultad fácil consistió en encontrar las vulnerabilidades que tenía un script de Python que implementaba el algoritmo de encriptación RSA (Rivest-Shamir-Adleman). El objetivo fue descifrar dos mensajes por medio de propiedades matemáticas y análisis criptográfico.

1.2 Hallazgos y recomendaciones

En general se encontraron dos vulnerabilidades, una relacionada con la debilidad de un método criptográfico y la otra con el manejo de claves públicas. El patrón de ataque identificado fue el análisis criptográfico de severidad muy alta que aprovecha vulnerabilidades para descifrar claves privadas e información. Se encontró un ataque de severidad media relacionado con un producto que aprovecha las vulnerabilidades identificadas para realizar un ataque con fuerza bruta para la factorización de la clave pública.

Las recomendaciones se resumen en:

- Usar una versión más fuerte del algoritmo RSA.
- Verificar y manejar las claves públicas de forma segura y correcta.
- Definir y usar estándares criptográficos, al igual que herramientas con seguridad comprobada.

2 Información general

El challenge resuelto para este write-up pertenece a la plataforma Hack the Box (HTB). Más detalles en la siguiente tabla.

Nombre	URL	Dificultad	Categoría	Fecha resolución	Username
RSaIsEasy	HTB	Easy	Crypto	30/04/2026	Kencast

Table 1: Información del challenge resuelto

3 Reconocimiento

3.1 Conexión y recopilación inicial de información

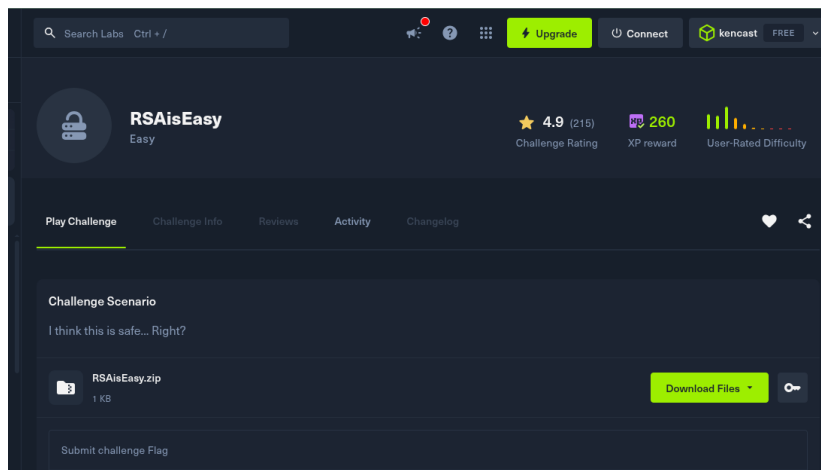


Figure 1: El challenge en HTB

Para iniciar el challenge hay que descargar el archivo `RSAisEasy.zip` en HTB a como se muestra en la figura 1.

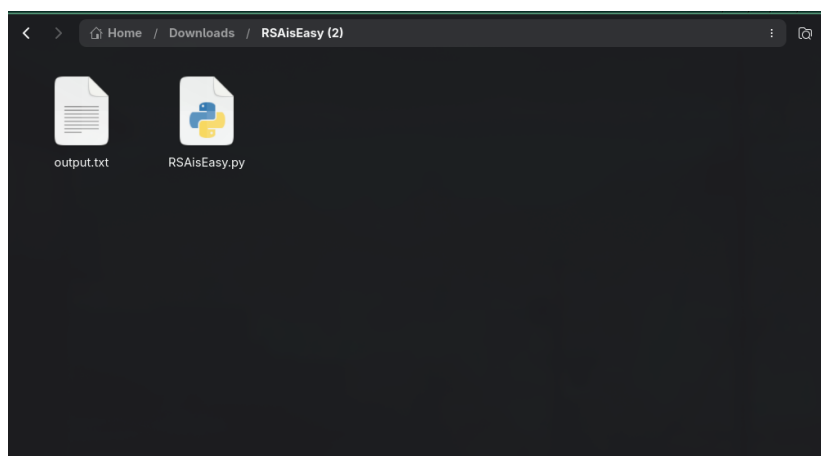


Figure 2: Archivos del challenge

Una vez descargado, hay que descomprimir el archivo zip, en HTB se encuentra la contraseña. En total se cuenta con dos archivos, uno de texto `output.txt` y un script del lenguaje Python `RSAisEasy.py`, como se observa en 2. Esto sug-

iere que para el challenge hay que analizar el código de Python y su salida en busca de vulnerabilidades. Como sugiere la categoría del challenge y su nombre, este está relacionado con el algoritmo RSA [1].

3.2 Elementos identificados

Como no se cuenta con ninguna red, computadora o sistema objetivo, no se utilizó ninguna herramienta para el análisis de puertos u otros elementos relacionados. Solo se cuenta con el script y la salida de este, así que los elementos identificados son los siguientes:

Elemento	Tipo	Descripción
bytes_to_long	Función	Pertenece al módulo de Python Crypto.Util.number. Convierte un string de bytes a un número. [2]
getPrime	Función	Pertenece al módulo de Python Crypto.Util.number. Retorna un número primo random de N bits. [2]
flag1, flag2	Variables	Hilera de bytes. Pertenece al módulo Secret, parece ser definido por el creador.
urandom	Función	Pertenece al módulo Os. Retorna una hilera de N bytes aleatorios.[3]
RSA	Algoritmo	Método de encriptación asimétrica muy usado [1].

Table 2: Elementos identificados en los archivos

Las bibliotecas que usan parecen ser seguras y aptas para procesos criptográficos, por lo que estas no son el enfoque. El script representa más que todo la implementación parcial del algoritmo RSA. En [4] hay una explicación más detallada del algoritmo, pero en general se resume en las fases de:

Generación de llaves:

Se eligen dos números primos grandes p y q , estos son secretos. Se calcula $n = pq$ como parte de la llave pública y privada. También, $\phi(n) = (p-1)(q-1)$ ya que p y q son números primos. Se elige $e : 1 < e < \phi(n) : \gcd(e, \phi(n)) = 1$, de esta

manera son coprimos. Finalmente, se calcula un d tal que $d * e \equiv 1 \mod \phi(n)$. La llave pública se representa por (n, e) y la privada por (n, d) .

Cifrado:

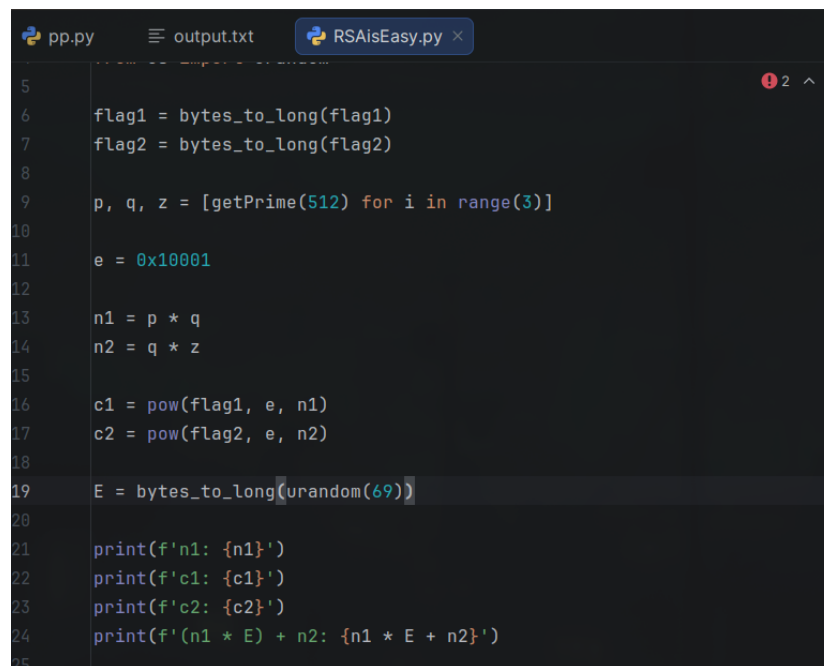
Si M es el mensaje representado de forma numérica. $C = M^e \mod n$ es el mensaje cifrado.

Descifrado:

El mensaje original es $M = C^e \mod n$.

4 Enumeración y análisis

4.1 Hallazgos

A screenshot of a code editor window with a dark theme. The window has three tabs: 'pp.py', 'output.txt', and 'RSAisEasy.py'. The 'RSAisEasy.py' tab is active and shows a Python script. The script is numbered from 5 to 25 on the left margin. It imports 'bytes_to_long' and 'urandom' from the 'secrets' module. Lines 6-7 convert 'flag1' and 'flag2' to long integers. Lines 9-10 generate three 512-bit primes 'p', 'q', and 'z'. Line 11 sets the public exponent 'e' to 0x10001. Lines 13-14 calculate moduli 'n1' and 'n2'. Lines 16-17 calculate ciphertexts 'c1' and 'c2'. Line 19 generates a random 69-byte message 'E'. Lines 21-24 print the modulus, ciphertexts, and the encrypted message. Line 25 is empty.

```
5
6     flag1 = bytes_to_long(flag1)
7     flag2 = bytes_to_long(flag2)
8
9     p, q, z = [getPrime(512) for i in range(3)]
10
11     e = 0x10001
12
13     n1 = p * q
14     n2 = q * z
15
16     c1 = pow(flag1, e, n1)
17     c2 = pow(flag2, e, n2)
18
19     E = bytes_to_long(urandom(69))
20
21     print(f'n1: {n1}')
22     print(f'c1: {c1}')
23     print(f'c2: {c2}')
24     print(f'(n1 * E) + n2: {n1 * E + n2}')
25
```

Figure 3: Parte del Código

En base a la explicación anterior, se puede entender mejor lo que hace el script de Python 3:

El objetivo es cifrar dos mensajes `flag1` y `flag2` importados del módulo `secrets`. Primero se generan tres números primos p , q y z todos de 512 bits

con la función `getPrime(512)`. Se define $e = 65537$ y dos llaves públicas: $(n1 = pq, e)$ y $(n2 = qz, e)$, una para cada mensaje respectivamente.

El cifrado de `flag1` y `flag2` es $c1 = \text{pow}(\text{flag1}, e, n1)$, $c2 = \text{pow}(\text{flag2}, e, n2)$. Por último, se genera un número aleatorio E de 69 bytes.

Lo que genera el programa como salida es $n1$, $c1$, $c2$ y $n1 * E + n2$. De esta forma, con los archivos se conoce la llave pública del mensaje 1, el mensaje 1 y 2 cifrados y una expresión extra.

La mayor parte del código implementa partes del algoritmo estándar. Lo que llama la atención es el cálculo de la expresión $n1 * E + n2$, ya que no es parte del algoritmo y también que no se comparte el valor $n2$. Además, para la llave se usan aproximadamente 1024 bits, lo cual ya no es recomendado por llegar a ser factorizable.

4.2 Posibles vectores de ataque

En HTB la bandera esperada corresponde a un texto, y tomando en cuenta la información que tenemos, el objetivo es descifrar algunos de los dos mensajes, o ambos. De acuerdo con el algoritmo, la única forma de poder hacer esto es conociendo d y n . Es suficiente con conocer p y q ya que $\phi(n) = (p-1)(q-1)$ y d es el inverso modular de e .

Los posibles vectores de ataque consisten en obtener la mayor cantidad posible de información a partir de la expresión $n1 * E + n2$, encontrar alguna condición numérica que no se cumpla o tratar de factorizar $n1$ para obtener $p1$ y $q1$. Sin embargo, esta última opción es poco factible por la cantidad de recursos computacionales necesarios.

4.3 Caminos probados

Primero se trató de verificar la condición de coprimalidad de e para obtener información de d , pero para esto es necesario conocer $\phi(n)$, así que no se logró nada.

Se decidió probar solo descifrar $c1$ por simplicidad, obteniendo los números asociados para esto. Sin embargo, también era necesario descifrar $c2$ para obtener la bandera, por lo que se tuvo que realizar de nuevo el procedimiento para $c2$.

5 Explotación

5.1 Vulnerabilidades identificadas

ID	Nombre	Descripción
CWE-326	Inadequate Encryption Strength	El producto transmite datos usando un método criptográfico fuerte teóricamente, pero vulnerable a ataques de fuerza bruta.
CWE-522	Insufficiently Protected Credentials	El producto transmite o guarda credenciales (o llaves) pero usa un método inseguro susceptible a la recuperación de estos.

Table 3: Principales vulnerabilidades encontradas en la fuente CWE [5]

ID	Nombre	Severidad	Descripción
CAPEC-97	Cryptanalysis	Muy alta	Encontrar vulnerabilidades en algoritmos criptográficos para descifrar un mensaje o la llave privada.

Table 4: Principales patrones de ataque encontrados en la fuente CAPEC [6]

ID	Nombre	Sys-	CVSS	Descripción
CVE-2026-5363	TP-Link tems Inc.		Medium	Encriptación de contraseñas usando RSA-1024. Un atacante puede interceptar el mensaje y hacer un ataque fuerza bruta o de factorización para encontrar la clave privada.

Table 5: Ataques relacionados encontrados en la plataforma CVE [7]

En la tabla 3 se identifican las principales vulnerabilidades relacionadas con el challenge resuelto. La vulnerabilidad **CWE-326** se aplica en este caso, ya que se identificó que se estaba usando RSA-1024 para encriptar los mensajes, esto vuelve vulnerable al algoritmo a un ataque de fuerza como la factorización de la clave pública para descifrar el mensaje. Sin embargo, para llevar a cabo esto se requiere mucha capacidad de cómputo. En la tabla 5 se identifica un ataque de severidad media que se relaciona con esta vulnerabilidad.

La siguiente vulnerabilidad **CWE-522** es la que permitió resolver este challenge debido a que por medio de una expresión matemática que se compartió, la cual involucraba las llaves públicas y gracias al hecho de que estas compartían un factor primo, se logró obtener el resto de parámetros para descifrar el mensaje.

En general, el patrón de ataque que aplica en este caso es **CAPEC-97**, especificado en la tabla 4. El algoritmo RSA no es el que tenía la vulnerabilidad, sino la forma en la que se manejan y se comparten las llaves, lo que permite obtener información clave de los parámetros del algoritmo. Por otro lado, no se encontró un ataque específico (CVE) que represente la forma en que se resolvió este challenge, por lo que se trata más de una vulnerabilidad en un caso particular.

Respecto a la relación con los contenidos del curso, estas vulnerabilidades y patrones de ataques corresponden al reino de "Criptografía" vistos en la sección "2.1. Taxonomía de los patrones de Ataque" relacionado con el algoritmo RSA. El patrón de ataque consistió en el análisis matemático y criptográfico para obtener información del algoritmo.

5.2 Pasos para explotación

Analizando la expresión que se imprime en el código 3:

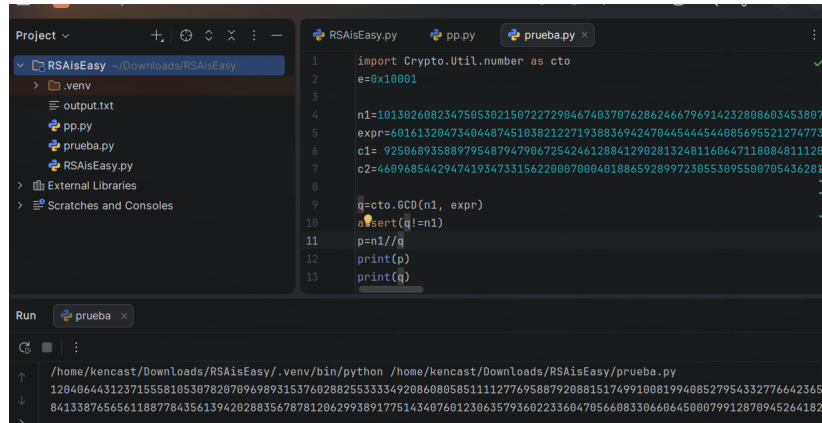
$$n1 * E + n2 = p * q * E + q * z = q * (p * E + z) \quad (1)$$

Con esto se puede notar que esta expresión es divisible por q , porque de alguna forma podríamos relacionar esta expresión con $n1$ para encontrar q . Si calculamos el máximo común divisor (mcm):

$$\text{mcm}(n1, n1 * E + n2) = \text{mcm}(p * q, q * (p * E + z)) \quad (2)$$

De esta forma podemos concluir que la ecuación anterior tiene solo dos posibles resultados: q o $p * q$. Debido a que p y q son números primos y q divide a ambos valores. Por otro lado, la única forma de que el resultado sea $p * q$ es si p divide a $p * E + z$, lo cual solo es posible si $p = z$, ya que z es un número primo.

Con los datos que se tienen se puede calcular el mcm para ver cuál es el resultado.



```
Project ▾
  ▾ RSAIsEasy ~/Downloads/RSAIsEasy
    ▾ .venv
    ▾ output.txt
    ▾ pp.py
    ▾ prueba.py
    ▾ RSAIsEasy.py
    ▾ External Libraries
    ▾ Scratches and Consoles

1 import Crypto.Util.number as cto
2 e=0x10001
3
4 n1=10138269823475853821587227298467483707628624667969142328086834538072
5 expr=601613284734844874518382122719388369424784454445448856955212747733
6 c1= 925068935889795487947986725424612884129828132481160647118084811286
7 c2=4689685442947419347331562288078004818865928997230553895508705436281
8
9 q=cto.GCD(n1, expr)
10 assert(q!=n1)
11 p=n1//q
12 print(p)
13 print(q)
```

Run prueba ×

/home/kencast/Downloads/RSAIsEasy/.venv/bin/python /home/kencast/Downloads/RSAIsEasy/prueba.py
12040644312371555810530782070969893153768288255333492086408058511127769588792088151749910881994085279543327764423658
841338765656118877843561394202883567878120629938917751434076812306357936822336047056608330660645080799128709452641820

Figure 4: Encontrando p y q

Primero se crea un programa de Python como se muestra en la figura 4. En este se importa la biblioteca `Crypto.Util.number` para hacer uso de funciones útiles para los cálculos con números grandes. Después se definen todos los números que se encontraron en los archivos. En la línea número 9 se calcula el $\text{mcm}(n1, \text{expr})$ y con la verificación en la línea 10 se comprueba que este corresponde a q , por lo que también se encuentra $p = n1/q$.

```

Project ▾
  RSAIsEasy ~/Downloads/RSAIsEasy
    .venv
    output.txt
    pp.py
    prueba.py
    RSAIsEasy.py
  External Libraries
  Scratches and Consoles

Run prueba ×

717652241588678642114435431346904221032860951957629033993588982610801706670890675169628982714844252198924130787175951
Process finished with exit code 0

```

Figure 5: Cálculo de $\phi(n1)$ y $d1$

Una vez encontrados p y q , se pueden encontrar el resto de parámetros necesarios para el descifrado del mensaje 1. En la figura 5 se calcula $\phi(n1) = (p-1)(q-1)$ y se verifica la condición de coprimalidad entre e y $\phi(n1)$. Finalmente, se puede encontrar $d1$ que corresponde al inverso modular de e módulo $\phi(n1)$, y es el número necesario para descifrar $c1$.

```

Project ▾
  RSAIsEasy ~/Downloads/RSAIsEasy
    .venv
    output.txt
    pp.py
    prueba.py
    RSAIsEasy.py
  External Libraries
  Scratches and Consoles

Run prueba ×

/home/kencast/Downloads/RSAIsEasy/.venv/bin/python /home/kencast/Downloads/RSAIsEasy/prueba.py
HTB{1_m1ght_h4v3_m3ss3d
Process finished with exit code 0

```

Figure 6: Descifrado del mensaje 1

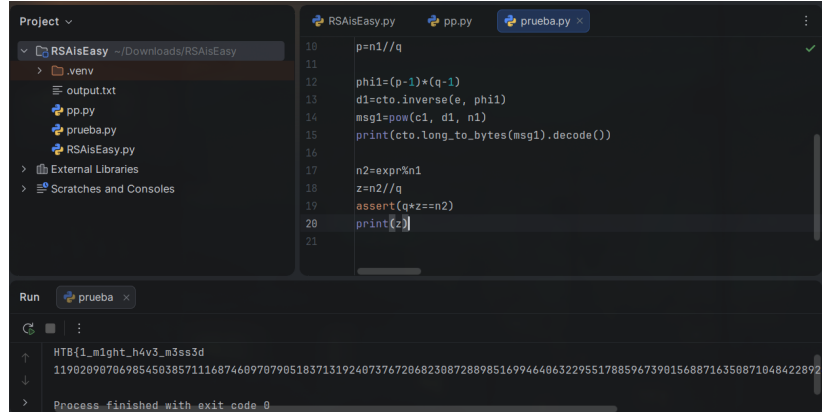
Finalmente, en 6 se descifra el mensaje 1 en la línea 15 como $msg1 = c1^{d1} \bmod n1$. Para obtener la representación en texto de este, se usa la función `long_to_bytes(msg1).decode()`. En la terminal se muestra parte del mensaje.

Para completar el challenge se ocupa también descifrar la otra parte del mensaje ($c2$), por lo que se ocupa el encontrar el valor de z . Volviendo a

analizar la siguiente expresión:

$$n1 * E + n2 \equiv n2 \pmod{n1} : n2 < n1 \quad (3)$$

$$z = n2/q \quad (4)$$



```

10 p=n1//q
11
12 phi1=(p-1)*(q-1)
13 d1=cto.inverse(e, phi1)
14 msg1=pow(c1, d1, n1)
15 print(cto.long_to_bytes(msg1).decode())
16
17 n2=expr%n1
18 z=n2//q
19 assert(q*z==n2)
20 print(z)
21

```

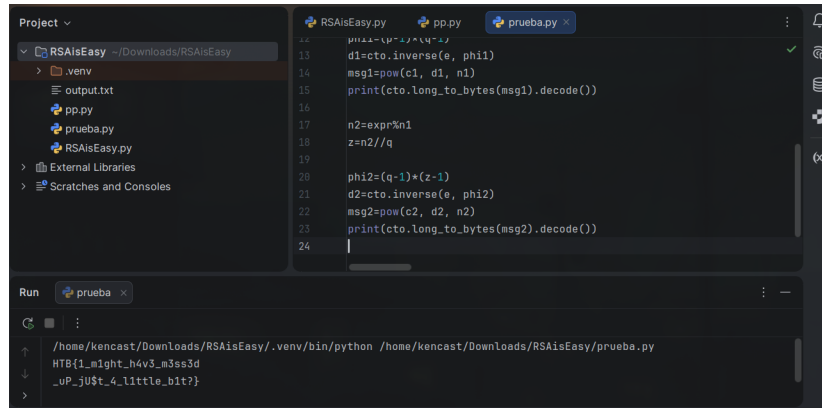
```

Run prueba x
HTB{1_m1ght_h4v3_m3ss3d
119029987869854503857111687460970798518371319240737672068230872889851699464063229551788596739015688716350871048422892
Process finished with exit code 0

```

Figure 7: Cálculo de z y $n2$

Por lo que se puede probar si se cumple que $n2 < n1$, con lo cual se obtendría z . En la figura 7 se observa el cálculo de $n2$ y z , y en la línea 19 se verifica que es correcto.



```

10 p=n1//q
11
12 phi1=(p-1)*(q-1)
13 d1=cto.inverse(e, phi1)
14 msg1=pow(c1, d1, n1)
15 print(cto.long_to_bytes(msg1).decode())
16
17 n2=expr%n1
18 z=n2//q
19
20 phi2=(q-1)*(z-1)
21 d2=cto.inverse(e, phi2)
22 msg2=pow(c2, d2, n2)
23 print(cto.long_to_bytes(msg2).decode())
24

```

```

Run prueba x
/home/kencast/Downloads/RSaIsEasy/.venv/bin/python /home/kencast/Downloads/RSaIsEasy/prueba.py
HTB{1_m1ght_h4v3_m3ss3d_uP_jU$t_4_l1ttle_b1t?}

```

Figure 8: Descifrado del mensaje 2

Una vez conociendo q y z , el descifrado del mensaje 2 es análogo al del mensaje 1, como se muestra en la figura 8. De esta forma se lograron descifrar ambos mensajes de manera correcta, como se muestra en la terminal. La bandera para este challenge corresponde a la concatenación de ambos mensajes.

6 Evidencia de resolución

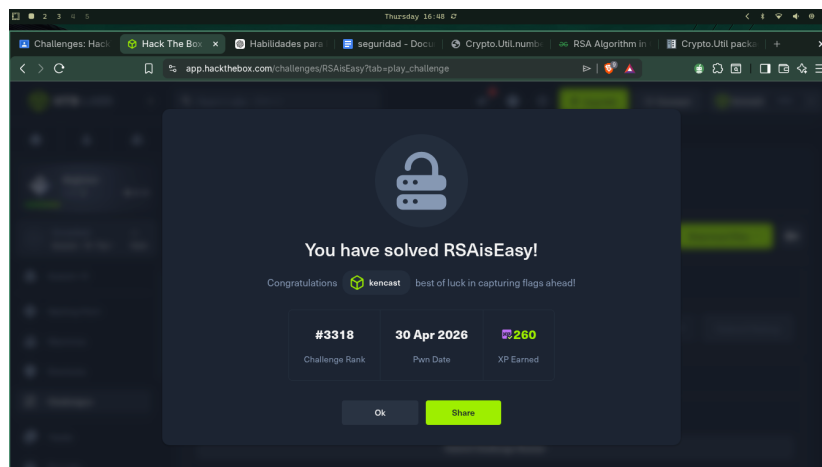


Figure 9: Resolución del challenge

7 Remediación

Estas vulnerabilidades se mitigarían en un entorno de producción real haciendo un cambio de RSA-1024 a RSA-2048 o más. RSA-1024 ya no es recomendado y se considera que se puede llegar a factorizar la clave pública con la capacidad de cómputo actual. Por otro lado, es necesario verificar que las claves que se generan para estos algoritmos sean seguras y lo suficientemente aleatorias. Principalmente, si se tienen varias llaves públicas hay que verificar que estas no compartan un mismo número primo en su factorización, ya que esto puede causar que se descubra la llave privada.

Finalmente, es esencial asegurarse de compartir solo la información necesaria relacionada con claves y parámetros de algoritmos criptográficos; dar más datos de lo necesario puede ocasionar que, por medio de propiedades matemáticas, se descubran nuevos valores.

Las buenas prácticas relacionadas con los principios de seguridad desde el diseño son las siguientes [8]:

- **Definir y usar estándares criptográficos:** es crítico usar los mecanismos criptográficos de forma correcta y segura, entendiendo cómo estos funcionan y definiendo las formas de proteger y manejar las llaves de cifrado y certificados. Esto incluye también usar bibliotecas que tengan una seguridad comprobada para los procedimientos criptográficos y matemáticos.

- **Requerir el uso de funciones y herramientas con seguridad comprobada:** asegurarse de que los equipos de desarrollo usen soluciones bien establecidas y de seguridad comprobada. Manteniéndose al día con los cambios y las versiones de las herramientas. Por ejemplo, usando una versión segura de RSA.

8 Lecciones aprendidas

Con este challenge pude profundizar y lograr una buena comprensión del algoritmo RSA, así como de las vulnerabilidades que este puede tener si no se hace de forma correcta, y también los estándares y prácticas recomendadas. En general, pude indagar un poco sobre los métodos criptográficos y prácticas que se usan para una transmisión segura de información. Lo que haría diferente en este challenge sería notar de forma más directa algunas propiedades matemáticas que permitían descifrar los mensajes.

9 Recursos consultados

References

- [1] Información sobre el algoritmo RSA: https://en.wikipedia.org/wiki/RSA_cryptosystem
- [2] Módulo de Python usado en el script: <https://www.pycrypto.org/api/current/Crypto.Util.number-module.html>
- [3] Módulo de Python: <https://zetcode.com/python/os-urandom/>
- [4] Explicación del algoritmo RSA: <https://www.geeksforgeeks.org/computer-networks/rsa-algorithm-cryptography/>
- [5] Common Weakness Enumeration: <https://cwe.mitre.org/index.html>
- [6] Common Attack Pattern Enumeration and Clasification: <https://capec.mitre.org/index.html>
- [7] Common Vulnerabilities and Exposures: <https://www.cve.org/About/Overview>
- [8] Principios de seguridad del software desde el diseño: <https://www.microsoft.com/en-us/securityengineering/sdl/practices>